HA Skript

Höhere Algorithmik – Design and Analysis of Algorithms

Winter Semester 2026/27

Sowas • Freie Universität Berlin

Stand: 26.10.2026 • Lectures of 16.10., 19.10., 22.10. and 26.10.2026

Contents

1 Introduction and Overview 3

What we measure and on which machine: RAM model, uniform vs. logarithmic cost, worst-case complexity, Landau symbols.

LECTURE 16.10.2026 . . . 3

1.1 Goals and Main Topics of the Class . . . 3

1.2 Computational Model . . . 4

1.3 Complexity . . . 6

1.4 Asymptotic Runtime as $n \to \infty$ . . . 7

1.5 O-Notation for Algorithms ($n \to \infty$) . . . 7

2 Divide and Conquer 9

Split, solve, combine: three ways to solve recurrences, Karatsuba multiplication, the master theorem with full proof, and linear-time selection.

LECTURE 19.10.2026 . . . 9

2.1 Divide and Conquer . . . 9

2.2 Solving Divide-and-Conquer Recurrences . . . 10

2.2.1 Method 1: Recursion Tree . . . 10

2.2.2 Method 2: Iterative Expansion . . . 10

2.2.3 Method 3: Proof by Induction . . . 11

2.3 Multiplication Methods . . . 11

2.3.1 School Method: $O(n^2)$ . . . 11

2.3.2 Divide and Conquer . . . 12

2.3.3 Karatsuba Algorithm . . . 12

2.3.4 Proof by Induction for Karatsuba . . . 13

2.4 The Master Theorem . . . 14

LECTURE 22.10.2026 . . . 14

2.5 Regularity Condition . . . 15

2.6 Not Covered: the Gap Between Case II and III . . . 15

2.7 Proof of the Master Theorem . . . 16

Step 1: The arguments must shrink . . . 16

Step 2: Positivity and the base . . . 16

Step 3: Sandwich . . . 16
Step 4: Upper bound via a comparison function $\hat{T}$ . . . 17
Step 5: Removing the ceiling by a shift . . . 17
Step 6: Solving (6) . . . 17
2.8 Shifting the Domain . . . 18
2.9 Shifting the Value . . . 18
LECTURE 26.10.2026 . . . 19
2.10 The Selection Problem . . . 19
2.11 Quickselect . . . 19
2.12 Randomized Pivot . . . 20
2.13 Deterministic Algorithm: Median of Medians . . . 20
Notes on Corrections . . . 22

Key definitions and results

Def. 1.1	RAM: registers, indirect addressing, jumps	4
Def. 1.2	Uniform and logarithmic cost model	5
Def. 1.3	Worst-case complexity $T(n)$, $S(n)$	6
Def. 1.5	Landau symbols $O, o, \Omega, \omega, \Theta$	7
Lemma 1.6	Limit criterion for o, Θ, ω	8
Thm. 2.2	Master theorem: $T(n) = aT(n/b) + f(n)$, three cases	14
Thm. 2.3	Median of medians: selection in $O(n)$	21

Chapter 1

Introduction and Overview

Lecture of 16.10.2026 • RAM model, cost models, complexity, O-notation

Before we design and compare algorithms, we fix *what* we measure and *on which machine*. This chapter introduces the RAM as computational model, the cost models, time and space complexity, and the O-notation, which compares runtimes independently of constants and small inputs.

1.1 Goals and Main Topics of the Class

The course has two sides. On the **design** side we learn general principles that work for many problems at once. On the **analysis** side we prove that an algorithm is correct and determine the resources it needs.

General design principles	**Analysis**
Divide and conquer	Correctness
Dynamic programming	Runtime
Greedy method	Space
Branch and bound	Lower bounds, NP-hardness / NP-completeness, reductions

- **Divide and conquer** splits a problem into smaller subproblems, solves them recursively and combines the partial solutions.
- **Dynamic programming** solves overlapping subproblems only once and stores their results in a table.
- **Greedy methods** build a solution step by step and always take the choice that looks best at the moment.
- **Branch and bound** searches all solutions systematically, but cuts off branches that provably cannot beat the best solution found so far.

All four principles are typically used to solve **optimization problems**.

The analysis also covers negative results. Sometimes no efficient algorithm is likely to exist (NP-hardness, NP-completeness), or every algorithm needs at least a certain time (lower bounds). The main tool is the **reduction**: it transforms every instance of a problem A into an instance of a problem B, so an efficient algorithm for B would also solve A. If A is hard, B must be hard as well.

1.2 Computational Model

"The runtime of an algorithm" only makes sense once we fix the machine. Candidates:

- **Actual machine** – too dependent on hardware, compiler and technology; results are not comparable.
- **Turing machine** – mathematically precise, but primitive: it moves step by step along a tape, so simple operations cost many steps.
- **RAM** (random-access machine) – realistic: it accesses any memory cell directly, like a real computer.

The RAM combines both advantages: it is precisely defined and close to real computers.

The RAM

Definition 1.1 (RAM). Storage: an unbounded sequence of registers $R_0, R_1, R_2, \ldots$; each register holds an integer. Commands:

- Arithmetic: $R_3 \leftarrow R_1$, $R_7 \leftarrow 15$, $R_2 \leftarrow R_2 + R_5$, with $+, -, \cdot, /$ (integer division $\lfloor x \rfloor$).
- Indirect addressing: $R_5 \leftarrow (R_0)$. If $R_0 = 100$, this copies the content of R_{100}.
- Unconditional jumps `GOTO B`; conditional jumps `IF` ... `GOTO B` (B is a label).
- Stop the machine: `HALT`.

Indirect addressing is the key feature: a register's content is used as an *address*. That is exactly a **pointer**, and it lets us access $A[i]$ when i is only known at runtime. Jumps give loops and case distinctions.

Remark (input convention). $R_0 = n$ = size of the input; $R_1, \ldots, R_n$ = the input itself.

Example (pointer).

```
     R5 ← 0
     R0 ← 10
A:   R5 ← R5 + (R0)
     R0 ← R0 + 1
     IF R0 < 20 GOTO A
```

This computes the sum $R_{10} + R_{11} + \cdots + R_{19}$ in R_5. R_0 runs as a pointer through the addresses 10 to 19; after ten passes $R_0 = 20$ and the loop stops.

Bounding the Register Contents

The primitive RAM is unrealistic: registers hold *arbitrarily large* integers, there are infinitely many registers, and every command costs the same. It is even **more powerful** than a real processor, which works with words of fixed length (e.g. 64 bits, -2^{63} to 2^{63}) and finite memory. Two fixes:

1) Word-RAM. Every register holds an integer with b bits: $-2^b < x < 2^b$. With indirect addressing a program can therefore address only 2^b registers.

Assumption: $b = c \cdot \log_2 n$ for a constant c, where n is the input size.

$$\Rightarrow\ 2^b = 2^{c \log_2 n} = n^c \text{ addressable registers (polynomially many)}$$

The word length grows only logarithmically with the input. A register must at least hold an address of the input (about $\log_2 n$ bits), but words stay small enough that one operation can cost one unit. $\rightarrow$ **Uniform cost model.**

2) Unbounded integers. Registers may hold arbitrarily large integers, but large numbers become more *expensive*.

Definition 1.2 (Cost models).

- **Uniform cost model** (Word-RAM): every step costs 1 unit.
- **Logarithmic cost model** (unbounded integers): an operation costs the sum of the logarithms of all involved addresses and operands, with

$$\ell(x) = \lceil \log_2 \max(2, |x|) \rceil .$$

$\ell(x)$ is essentially the number of bits of x. The $\max(2, |x|)$ makes every number cost at least 1, since $\log_2 0$ is undefined and $\log_2 1 = 0$.

Example. $R_0 = 100$, $R_{100} = -77$, $R_5 = 64$. Operation $R_5 \leftarrow (R_0) + R_5$, new content of R_5 is -13.

$$\text{cost} = \ell(5) + \ell(0) + \ell(100) + \ell(-77) + \ell(5) + \ell(64) + \ell(-13)$$

Each term is one number the operation touches: $\ell(5)$ the target address; $\ell(0)$ the address of R_0 (counts as $\log_2 2$); $\ell(100)$ the content of R_0, used as address; $\ell(-77)$ the operand in R_{100} (absolute value); $\ell(5)$ and $\ell(64)$ address and content of the second operand R_5; $\ell(-13)$ the result.

Remark (cheating). Without 1) or 2), several numbers $x_1, x_2, \ldots$ could be packed into one register R_5 and $y_1, y_2, \ldots$ into R_6. One addition $R_5 + R_6$ would then compute all sums $x_i + y_i$ at once, if enough space keeps carries apart. Both the word length bound and logarithmic costs prevent this.

R_5	x_3	x_2	x_1	$\cdots$
R_6	y_3	y_2	y_1	$\cdots$
$R_5{+}R_6$	$x_3{+}y_3$	$x_2{+}y_2$	$x_1{+}y_1$	$\cdots$

In this course we mostly use the **uniform cost model** (Word-RAM).

Example (Gaussian elimination). Solve an $n \times n$ system $Ax = b$ with integers of at most m bits.

- $O(n^3)$ steps in the uniform cost model.
- Rational arithmetic: $\frac{a}{b} + \frac{c}{d} = \frac{ad+cb}{bd}$.
- Logarithmic cost model – important: **reduce** the fractions. Then all numbers have $O(n(m + \log n))$ bits; without reducing, their length can grow exponentially in n.
- $\Rightarrow$ Runtime in the logarithmic cost model:

$$O\big(n^3 \cdot n(m + \log n)\big) = O\big(n^4(m + \log n)\big)$$

Reduced, all intermediate values are quotients of subdeterminants of A, which is why their length stays bounded. The gcd steps for reducing are not counted separately here.

1.3 Complexity

Runtime T = number of steps; **space** S = number of registers used. Both depend on the input size n, possibly on several parameters $n, m, \ldots$ (e.g. a graph with n vertices and m edges).

Definition 1.3 (Worst-case complexity – typical for analysis).

- $T(n)$ = maximal runtime over all inputs of size $\leq n$
- $S(n)$ = the same for space

The worst case gives a **guarantee**: no input of size at most n needs more than $T(n)$ steps. Taking all inputs of size *at most* n makes T monotonically increasing.

Another possible analysis is the **average-case complexity** – only mentioned. It needs a probability distribution over the inputs, which is often unknown in practice.

1.4 Asymptotic Runtime as $n \to \infty$

Exact runtime functions are complicated and their constants depend on the model. What matters is how the runtime *grows* for large inputs.

$T(n)$	$n = 1$	$n = 10$	$n = 100$	Class
$3n^2 + 1000$	1003	1300	**31 000**	$\Theta(n^2)$
$n^3 + 20$	21	**1021**	1 000 020	$\Theta(n^3)$
$2 \cdot 2^n$	**4**	2048	$2 \cdot 2^{100}$	$\Theta(2^n)$

Bold = winner for that n. For very small n even $2 \cdot 2^n$ wins; at $n = 10$, $n^3 + 20$ wins. From $n = 12$ on, $3n^2 + 1000$ is smaller and the gap keeps growing. For $n \to \infty$ the winner is $\Theta(n^2)$: it is **asymptotically preferred**, which is not necessarily the best choice in a practical application.

1.5 O-Notation for Algorithms ($n \to \infty$)

Let $f, g, h, \cdots : \mathbb{N}_0 \to \mathbb{R}$. Only large n matter, so we require positivity only from some point on.

Definition 1.4 (almost everywhere positive, AEP). f is AEP if there is an n_0 with $f(n) > 0$ for all $n \geq n_0$.

Example: $\log_2(\log_2 n)$ is undefined for $n \leq 1$ and 0 for $n = 2$, but positive for $n \geq 3$, so it is AEP with $n_0 = 3$.

For AEP functions we write $f \leq_{\text{AE}} g$ if there is an n_0 with $f(n) \leq g(n)$ for all $n \geq n_0$ – "f is at most g, except for finitely many n".

Definition 1.5 (Landau symbols).

$$
\begin{aligned}
O(f(n)) &= \{\, g \text{ AEP} \mid \exists\, c > 0 : g \leq_{\text{AE}} c \cdot f \,\} && \text{upper bound} \\
o(f(n)) &= \{\, g \text{ AEP} \mid \forall\, c > 0 : g \leq_{\text{AE}} c \cdot f \,\} && \text{strict upper bound} \\
\Omega(f(n)) &= \{\, g \text{ AEP} \mid \exists\, c > 0 : g \geq_{\text{AE}} c \cdot f \,\} && \text{lower bound} \\
\omega(f(n)) &= \{\, g \text{ AEP} \mid \forall\, c > 0 : g \geq_{\text{AE}} c \cdot f \,\} && \text{strict lower bound} \\
\Theta(f(n)) &= O(f(n)) \cap \Omega(f(n))
\end{aligned}
$$

- $g \in O(f)$: g grows *at most* as fast as f, up to a constant factor.
- $g \in o(f)$: for every $c > 0$, g eventually stays below $c \cdot f$ – g grows *strictly slower*.
- Ω and ω are the corresponding lower bounds.
- $g \in \Theta(f)$: g grows *exactly* as fast as f, up to constant factors.

Although $O(f(n))$ is a set, one writes $g(n) = O(f(n))$, e.g. $3n^2 + 1000 = O(n^2)$ and $4n^2 = O(n^2)$. This is a **one-way equation**, read only left to right: it does *not* follow that $3n^2 + 1000 = 4n^2$. Likewise $n^2 + O(n)$ means n^2 plus some function from $O(n)$.

Upper and lower bounds are two views of one relation: $g(n) = O(f(n)) \Leftrightarrow f(n) = \Omega(g(n))$, since $g \leq_{\text{AE}} c \cdot f$ is equivalent to $f \geq_{\text{AE}} \frac{1}{c} \cdot g$.

Lemma 1.6. Let f, g be AEP and $\lim_{n\to\infty} f(n)/g(n) = C$. Then

$$C \begin{cases} = 0 & \Rightarrow f(n) \in o(g(n)) \\ \in (0, \infty) & \Rightarrow f(n) \in \Theta(g(n)) \\ = \infty & \Rightarrow f(n) \in \omega(g(n)) \end{cases}$$

Examples: $(3n^2 + 1000)/n^2 \to 3$, so $3n^2 + 1000 \in \Theta(n^2)$; $n^3/2^n \to 0$, so $n^3 \in o(2^n)$. If the limit does not exist, the lemma says nothing.

Remark (Beware: Taylor formula). $f(x + h) = f(x) + h \cdot f'(x) + O(h^2)$ as $h \to 0$. In analysis, O describes the behaviour for *small* h. In this course, O always refers to $n \to \infty$ unless stated otherwise.

Chapter 2

Divide and Conquer

Lecture of 19.10.2026 • Divide and conquer, solving recurrences, Karatsuba multiplication, master theorem (continued 22.10.), selection (26.10.)

Divide and conquer is the first general design principle. Its runtime is naturally described by a **recurrence**, an equation that expresses $T(n)$ through T of smaller arguments. We learn three methods to solve recurrences, apply the principle to multiplying large numbers, and state the master theorem, which solves a whole class of recurrences at once.

2.1 Divide and Conquer

Divide and conquer is a **paradigm**: an algorithm works in three steps.

1. **Split** the problem into several smaller subproblems.
2. **Solve** the subproblems recursively.
3. **Combine** the solutions of the subproblems.

Example (sorting).

Step	Quicksort	Mergesort
1. Split	Choose a pivot a; split into lists $< a$ and $> a$: $O(n)$	Split into 2 lists of length $n/2$
2. Solve	Sort both lists recursively	Sort both lists recursively
3. Combine	Put together: $[< a \mid a \mid > a]$	Merge the two sorted lists: $O(n)$

Quicksort, step 3: $< a$ | a | $> a$

Quicksort does its work *before* the recursion, so combining is trivial. Mergesort does it *after* the recursion, so splitting is trivial. For Mergesort the runtime satisfies the recurrence

$$T(n) = 2\,T\left(\tfrac{n}{2}\right) + O(n) = O(n \log n).$$

For odd n the halves differ by one element:

$$T(n) = T\left(\lceil \tfrac{n}{2} \rceil\right) + T\left(\lfloor \tfrac{n}{2} \rfloor\right) + O(n) = O(n \log n).$$

The term $2\,T(n/2)$ stands for the two recursive calls (step 2), $O(n)$ for splitting and merging (steps 1 and 3). The factor in front is the **number of subproblems**; the check is: number of parts $\times$ size of a part $= n$ (here $2 \cdot n/2 = n$). The n in the formula is the size of the *current* call, so the same rule applies on every level.

Definition 2.1 (Bottom of the recursion). $T(n) \le C_0$ for $n \le N_0$.

Small inputs are solved directly. There are only finitely many inputs of size at most N_0, so their runtime is bounded by a constant C_0. Without this base case, T would not be determined.

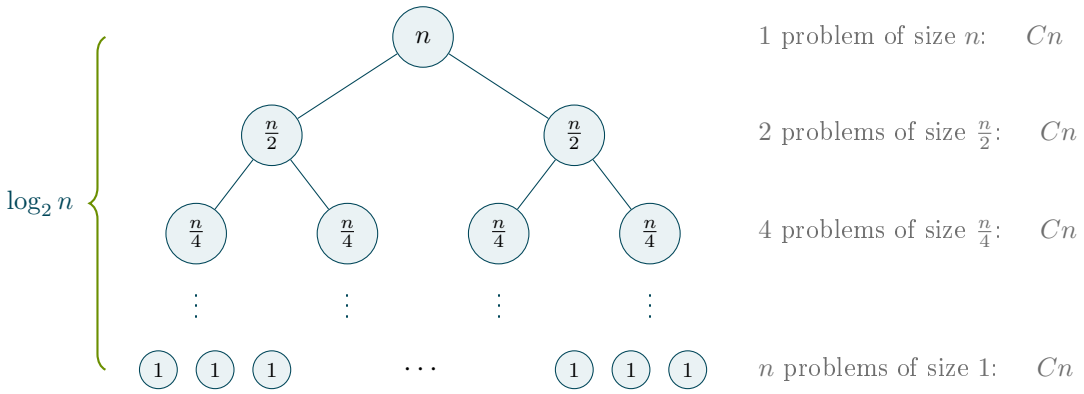
Worked example (merging costs exactly n, $T(1) = 0$): $T(2) = 2$, $T(4) = 8$, $T(8) = 24$, $T(16) = 64$ – always $n \cdot \log_2 n$.

2.2 Solving Divide-and-Conquer Recurrences

We show in three ways that $T(n) \leq 2\,T(n/2) + Cn$ gives $T(n) = O(n \log n)$. Cn replaces $O(n)$ by an explicit bound. To keep the calculations simple, $n = 2^k$.

2.2.1 Method 1: Recursion Tree

Every recursive call is a node, labelled with the work it does *itself* (without its subcalls).



Level	Problems	Size each	Work on level
0	1	n	Cn
1	2	$n/2$	Cn
2	4	$n/4$	Cn
i	2^i	$n/2^i$	Cn
$\log_2 n$	n	1	Cn

Every level costs $O(n)$. The size halves per level, so after $\log_2 n$ halvings we reach size 1.

$$\Rightarrow\ T(n) = O(n) \cdot (\log_2 n + 1) = O(n \log n)$$

2.2.2 Method 2: Iterative Expansion

Insert the recurrence into itself again and again:

$$\begin{aligned} T(n) &\leq Cn + 2\,T\left(\tfrac{n}{2}\right) \\ &\leq Cn + 2\left(C\tfrac{n}{2} + 2\,T\left(\tfrac{n}{4}\right)\right) \\ &\leq Cn + 2\left(C\tfrac{n}{2} + 2\left(C\tfrac{n}{4} + 2\,T\left(\tfrac{n}{8}\right)\right)\right) \\ &\ \ \vdots \\ &\leq Cn + 2C\tfrac{n}{2} + 4C\tfrac{n}{4} + \cdots + 2^{\log_2 n} \cdot C \cdot 1 \\ &= Cn\,(\log_2 n + 1) \end{aligned}$$

Every bracket contributes exactly Cn, one per level of the recursion tree. For the last term choose $C \geq C_0$, so $T(1) \leq C \cdot 1$. The expansion is the recursion tree written as a calculation.

2.2.3 Method 3: Proof by Induction

Methods 1 and 2 *find* the solution; induction *proves* it. Given:

$$T(n) \leq 2\,T\left(\tfrac{n}{2}\right) + Cn \quad \text{for } n > N_0, \qquad T(n) \leq C_0 \quad \text{for } n \leq N_0$$

The two lines cover separate ranges of n – a case distinction, not a contradiction. Simplifying assumption: $n = 2^k$.

> Claim. $T(n) \leq A \cdot n \log_2 n$ for all $n \geq 2$, for a suitable constant A.

Induction step ($n > N_0$):

$$\begin{aligned} T(n) &\leq 2\,T\left(\tfrac{n}{2}\right) + Cn \overset{\text{I.H.}}{\leq} 2\left(A \cdot \tfrac{n}{2} \cdot \log_2 \tfrac{n}{2}\right) + Cn \\ &= An \log_2 n - An + Cn \\ &\leq An \log_2 n \qquad \text{as long as } A \geq C \end{aligned}$$

Induction base ($2 \leq n \leq N_0$): $T(n) \leq C_0 \overset{!}{\leq} A \cdot n \log_2 n$. The right side is smallest for $n = 2$: $C_0 \leq 2A$, i.e. $A \geq C_0/2$.

With $\boldsymbol{A = \max(C, C_0/2)}$ both parts hold. ■

The key is the term $-An$: halving n reduces $\log_2 n$ by exactly 1, and this saving pays for the extra work Cn when $A \geq C$. The base starts at $n = 2$ because $n \log_2 n = 0$ for $n = 1$. We may assume $N_0 \geq 3$ (otherwise enlarge N_0); then $n > N_0$ implies $n/2 \geq 2$, so the induction hypothesis always applies.

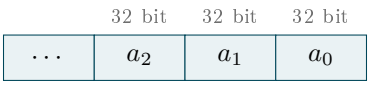
2.3 Multiplication Methods

In the logarithmic cost model, multiplying two large integers is not free: its cost grows with their length.

2.3.1 School Method: $O(n^2)$

Processors multiply 32-bit numbers directly in hardware. Larger numbers (e.g. 1000 bits) are split into 32-bit blocks; each block is one digit to the base $B = 2^{32}$.

$$A = a_0 + a_1 B + a_2 B^2 + \cdots + a_{n-1} B^{n-1}, \qquad 0 \leq a_i < B$$



High-school method: form the n^2 products of every digit of x with every digit of y and add them with the correct shifts.

Example ($345 \cdot 986$, base 10, $n = 3$).

	$\cdot 6$	$\cdot 80$	$\cdot 900$
5	30	400	4 500
40	240	3 200	36 000
300	1 800	24 000	270 000

$n^2 = 9$ terms; their sum is $345 \cdot 986 = 340\,170$. Each digit product is one word operation, and adding the shifted results also costs $O(n^2)$.

2.3.2 Divide and Conquer

Let x, y be n-bit numbers (n even for simplicity). Split each into a high and a low half of $n/2$ bits:



$$x = x_H \cdot 2^{n/2} + x_L, \qquad y = y_H \cdot 2^{n/2} + y_L$$
$$x \cdot y = x_H y_H \cdot 2^n + (x_H y_L + x_L y_H) \cdot 2^{n/2} + x_L y_L$$

One $n \times n$ multiplication becomes **four** $(n/2) \times (n/2)$ multiplications plus additions and shifts in $O(n)$:

$$\begin{aligned} T(n) &= 4\,T\left(\tfrac{n}{2}\right) + O(n) \\ &\le Cn + 2Cn + 4Cn + \cdots + 4^k\,T(1) = Cn\,(2^k - 1) + 4^k\,T(1) = O(n^2) \end{aligned}$$

since $4^k = (2^k)^2 = n^2$. The work doubles per level, so the n^2 base cases dominate. Divide and conquer alone gives no improvement: we still compute all n^2 digit products. To gain something, we must **save multiplications**.

2.3.3 Karatsuba Algorithm

Compute the middle term without both products separately:

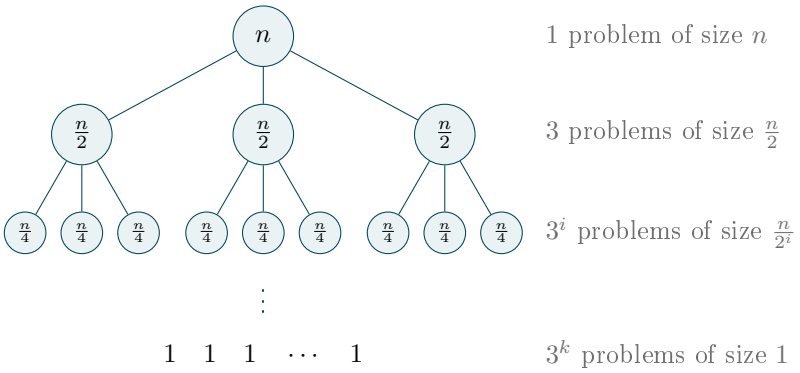
$$\begin{aligned} a &:= (x_H + x_L)(y_H + y_L) = x_H y_H + x_H y_L + x_L y_H + x_L y_L \\ b &:= x_H y_H, \qquad c := x_L y_L \\ d &:= a - b - c = x_H y_L + x_L y_H \\ \Rightarrow\ x \cdot y &= b \cdot 2^n + d \cdot 2^{n/2} + c \end{aligned}$$

Only **three** half-length multiplications (a, b, c); everything else is $O(n)$:

$$T(n) = 3\,T\left(\tfrac{n}{2}\right) + O(n)$$

Remark. $x_H + x_L$ and $y_H + y_L$ can have $n/2 + 1$ bits. This extra bit does not change the asymptotic runtime.

Recursion tree ($n = 2^k$):



Level i has 3^i problems of size $n/2^i$, with work $3^i \cdot C \cdot n/2^i = Cn \cdot (3/2)^i$. The work grows by the factor $3/2$ per level – a geometric series dominated by its last term, the 3^k base cases:

$$T(n) = O(3^k), \qquad 3^k = 3^{\log_2 n} = \left(2^{\log_2 n}\right)^{\log_2 3} = n^{\log_2 3} < n^2, \quad \log_2 3 \approx 1.585$$

Saving one multiplication per level improves $O(n^2)$ to $O(n^{1.585})$.

2.3.4 Proof by Induction for Karatsuba

Recurrence $T(n) \le 3\,T(n/2) + Cn$, with $\gamma := \log_2 3$.

First attempt: claim $T(n) \le A \cdot n^\gamma$.

$$T(n) \le 3\,T\left(\tfrac{n}{2}\right) + Cn \overset{\text{I.H.}}{\le} 3\,\frac{An^\gamma}{2^\gamma} + Cn \overset{?}{\le} An^\gamma$$

The main term needs $3A/2^\gamma \le A$, i.e. $3 \le 2^\gamma \Leftrightarrow \log_2 3 \le \gamma$ – this is where the exponent comes from. But for $\gamma = \log_2 3$ exactly, $3A/2^\gamma = A$ and the term Cn is left over. The induction fails, although the claim is true.

Idea: strengthen the claim to $T(n) \le A \cdot n^\gamma - Dn$.

$$\begin{aligned} T(n) &\le 3\,T\left(\tfrac{n}{2}\right) + Cn \overset{\text{I.H.}}{\le} 3\left(A\frac{n^\gamma}{2^\gamma} - D\frac{n}{2}\right) + Cn \\ &= An^\gamma - \tfrac{3}{2}Dn + Cn = An^\gamma - Dn - \left(\tfrac{D}{2} - C\right)n \\ &\le An^\gamma - Dn \qquad \text{as long as } D \ge 2C \end{aligned}$$

For the base $1 \le n \le N_0$ choose $A \ge C_0 + D$; this works because $n^\gamma \ge n \ge 1$. A stronger claim gives a stronger induction hypothesis: the recursive calls deliver the saving $\frac{3}{2}Dn$, which pays for Cn. Hence $T(n) = O(n^{\log_2 3})$.

2.4 The Master Theorem

The three methods work for every recurrence, but each time we calculate again. For divide-and-conquer recurrences there is a general result.

Theorem 2.2 (Master theorem). Let $a \geq 1$, $b > 1$, and let $f(n), T(n)$ be AEP with

$$a\,T\big(\lfloor \tfrac{n}{b} \rfloor\big) + f(n) \;\leq\; T(n) \;\leq\; a\,T\big(\lceil \tfrac{n}{b} \rceil\big) + f(n) \quad \text{for } n \geq n_0.$$

Critical exponent $\boldsymbol{\gamma := \log_b a \geq 0}$.

Case	Condition on $f(n)$	Result	Who dominates
1 ($<$)	$f(n) = O(n^{\gamma-\varepsilon})$ for some $\varepsilon > 0$	$T(n) = \Theta(n^{\gamma})$	Leaves
2 ($=$)	$f(n) = \Theta(n^{\gamma})$	$T(n) = \Theta(n^{\gamma} \log n)$	Every level
3 ($>$)	$f(n) = \Omega(n^{\gamma+\varepsilon})$ for some $\varepsilon > 0$	$T(n) = \Theta(f(n))$	Root

- a = number of subproblems, b = factor by which the size shrinks, $f(n)$ = work for splitting and combining.
- $\lceil \cdot \rceil$ and $\lfloor \cdot \rfloor$ allow n not divisible by b (like odd n in Mergesort); rounding does not change the result.
- The recursion tree has $a^{\log_b n} = n^{\gamma}$ leaves. The theorem compares $f(n)$, the work at the root, with n^{γ}, the work at the leaves. In Case 1 the work grows per level, in Case 2 every level costs the same, in Case 3 it shrinks per level.

Remark (Case 3). The usual formulation also requires a **regularity condition**: $a \cdot f(n/b) \leq c \cdot f(n)$ for a constant $c < 1$ and all large n. For polynomials $f(n) = n^p$ with $p > \gamma$ it holds automatically.

Examples.

Algorithm	a	b	γ	$f(n)$	Case	$T(n)$
Mergesort	2	2	1	$\Theta(n)$	2	$\Theta(n \log n)$
Simple D&C multiplication	4	2	2	$\Theta(n)$	1	$\Theta(n^2)$
Karatsuba	3	2	$\log_2 3 \approx 1.585$	$\Theta(n)$	1	$\Theta(n^{\log_2 3})$

For Karatsuba, Case 1 applies with $\varepsilon = \log_2 3 - 1 \approx 0.585 > 0$. All three confirm the results calculated by hand.

Lecture of 22.10.2026 • Regularity condition, proof, shifting the domain and the value

Section 2.4 stated the master theorem. The following sections add the regularity condition, show a recurrence the theorem does not cover, prove it, and use the same shifting idea to bring messy recurrences into the right form. From here on the gap is called δ, and the cases are named I ($-$), II ($=$) and III ($+$) as in the notes; they are Cases 1–3 of Theorem 2.2.

For the proof we number the two inequalities of Theorem 2.2, with $f(n), T(n) > 0$ for $n \geq n_0$:

$$(1) \quad T(n) \leq a\,T\big(\lceil \tfrac{n}{b} \rceil\big) + f(n)$$
$$(2) \quad T(n) \geq a\,T\big(\lfloor \tfrac{n}{b} \rfloor\big) + f(n)$$

2.5 Regularity Condition

Case III also holds if

$$\frac{a\, f\left(\left\lceil \frac{n}{b} \right\rceil\right)}{f(n)} \leq c < 1 \quad \text{for } n \geq n_0.$$

The work at the root then dominates by a constant factor per level, so the level costs form a decreasing geometric series. This covers non-polynomial f such as 2^n or $2^{\sqrt{n}}$.

The regularity condition alone is enough for Case III: applying $f(n) \geq \frac{a}{c} f(n/b)$ repeatedly gives $f(n) = \Omega(n^{\gamma+\delta})$ with $\delta = \log_b(1/c) > 0$. For a polynomial $f(n) = n^{\gamma+\delta}$ it holds automatically with $c = b^{-\delta} < 1$. Only with the weaker assumption $f = \Omega(n^{\gamma+\delta})$ alone must it be checked in addition (textbook form, e.g. CLRS).

Example. $f(n) = 2^n$, $a = b = 2$. With $\lceil n/2 \rceil \leq (n+1)/2$:

$$\frac{2 \cdot 2^{\lceil n/2 \rceil}}{2^n} \leq \frac{2 \cdot 2^{(n+1)/2}}{2^n} = 2^{(3-n)/2} \leq \tfrac{1}{2} \quad \text{for } n \geq 5$$

So $c = 1/2$ and $T(n) = \Theta(2^n)$.

2.6 Not Covered: the Gap Between Case II and III

$$T(n) = 2\,T\left(\tfrac{n}{2}\right) + \Theta(n \log_2 n), \qquad a = b = 2,\ \gamma = 1$$

f grows faster than $n^\gamma = n$, but not by a polynomial factor n^δ, and the regularity condition fails (the ratio tends to 1). No case applies. The recursion tree gives the answer: about $\log_2 n$ levels, each costing up to $n \log n$, so $T(n) = \Theta(n \log^2 n)$.

Remark (Extended Case II). Many textbooks close part of this gap: if $f(n) = \Theta(n^\gamma \log^k n)$ with $k \geq 0$, then $T(n) = \Theta(n^\gamma \log^{k+1} n)$. Our example is $k = 1$. For $k = -1$ one gets $\Theta(n^\gamma \log\log n)$, for $k < -1$ $\Theta(n^\gamma)$. The version from the lecture does not contain this extension.

2.7 Proof of the Master Theorem

Step 1: The arguments must shrink

We may always enlarge n_0. Then we can assume that (1) leads to a smaller argument on the right side. This is not automatic: for $b = 1.1$ and $n = 5$, $\lceil 5/1.1 \rceil = \lceil 4.55 \rceil = 5$, so $T(5)$ would depend on itself.

$$\left\lceil \tfrac{n}{b} \right\rceil < \tfrac{n}{b} + 1 \leq n \iff n \geq \frac{b}{b-1}$$

Increase n_0 if necessary so that $n_0 \geq b/(b-1)$.

Mixed recursions. Mergesort has $T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + f(n)$, which is neither (1) nor (2) with equality. **Remark:** if f is monotone, $f(n+1) \geq f(n)$, this can be reduced to (1) and (2) (Step 3).

Step 2: Positivity and the base

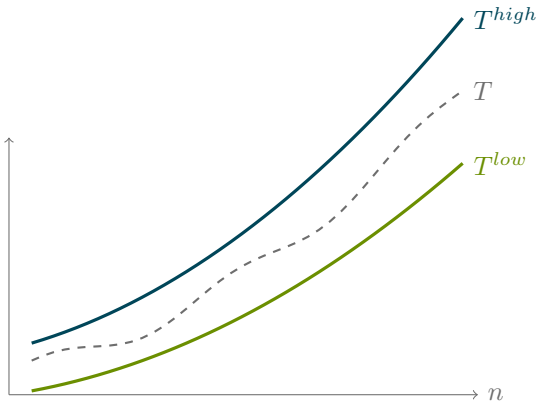
- $f(n), T(n) > 0$ for $n \geq n_0/b - 1$, by enforcing a bigger n_0 if necessary. The smallest argument $\lfloor n/b \rfloor \geq n/b - 1$ then stays in the positive range.
- (3) $T(n) \leq M$ for $n < n_0$, where M is the maximum of finitely many values.

Step 3: Sandwich

Define two clean recursions:

- $\boldsymbol{T^{high}}$ by (1) and (3) with equality,
- $\boldsymbol{T^{low}}$ by (2) with equality and $T^{low}(n) = 0$ for $n < n_0$.

> Claim. T with the mixed recursion is sandwiched: $T^{low}(n) \leq T(n) \leq T^{high}(n)$. If f is monotone, then T^{high} is monotone.



Proof of $T(n) \leq T^{high}(n)$ by induction (Mergesort case, $T^{high}(n) = 2T^{high}(\lceil n/2 \rceil) + f(n)$).

Base $n < n_0$: by (3), $T(n) \leq M = T^{high}(n)$.

Step:

$$\begin{aligned} T(n) &\le T\big(\lfloor \tfrac{n}{2} \rfloor\big) + T\big(\lceil \tfrac{n}{2} \rceil\big) + f(n) \\ &\overset{\text{I.H.}}{\le} T^{high}\big(\lfloor \tfrac{n}{2} \rfloor\big) + T^{high}\big(\lceil \tfrac{n}{2} \rceil\big) + f(n) \\ &\le 2\,T^{high}\big(\lceil \tfrac{n}{2} \rceil\big) + f(n) = T^{high}(n) \qquad (T^{high} \text{ monotone}) \end{aligned}$$

The master theorem applies to T^{high} and T^{low}. Both give the same $\Theta(\dots)$, so $T(n) = \Theta(\dots)$ as well.

Step 4: Upper bound via a comparison function $\hat{T}$

Let $f(n) = \Theta(n^\ell) \le u \cdot n^\ell$ for $n \ge n_0$, with $\ell = \gamma + \delta$, γ or $\gamma - \delta$. **Goal:** find $\hat{T}$ with

$$\begin{aligned} &(4) \quad \hat{T}(n) \ge a\,\hat{T}\big(\lceil \tfrac{n}{b} \rceil\big) + u\,n^\ell \quad \text{for } n \ge n_0 \\ &(5) \quad \hat{T}(n) \ge M \qquad\qquad\qquad\quad \text{for } \tfrac{n_0}{b} \le n < n_0 \end{aligned}$$

> Claim. Then $T(n) \le \hat{T}(n)$.

Proof by induction. $n < n_0$ ✓ by (3) and (5). For $n \ge n_0$:

$$T(n) \le a\,T\big(\lceil \tfrac{n}{b} \rceil\big) + f(n) \overset{\text{I.H.}}{\le} a\,\hat{T}\big(\lceil \tfrac{n}{b} \rceil\big) + u\,n^\ell \overset{(4)}{\le} \hat{T}(n)$$

Step 5: Removing the ceiling by a shift

Let T' be defined on the real numbers, monotone, and a solution of

$$(6) \quad T'(x) = a\,T'\big(\tfrac{x}{b}\big) + u'\,x^\ell$$

Set $\boldsymbol{\hat{T}(n) := T'(n - v)}$ with $\boldsymbol{v = b/(b-1)}$. Then $v - v/b = 1$, so $(n-v)/b = n/b + 1 - v$, and

$$\begin{aligned} \hat{T}(n) = T'(n-v) &= a\,T'\big(\tfrac{n-v}{b}\big) + u'(n-v)^\ell \\ &= a\,T'\big(\tfrac{n}{b} + 1 - v\big) + u'(n-v)^\ell \\ &\ge a\,\hat{T}\big(\lceil \tfrac{n}{b} \rceil\big) + u\,n^\ell \end{aligned}$$

The last step uses $\lceil n/b \rceil \le n/b + 1$ and monotonicity of T', and $u'(n-v)^\ell \ge u\,n^\ell$ for u' big enough. So $\hat{T}$ satisfies (4).

Step 6: Solving (6)

Idea: $T'(n) = Dn^\gamma + En^\ell$. Substitute into (6):

$$\begin{aligned} Dn^\gamma + En^\ell &\overset{!}{=} a\left(D\left(\tfrac{n}{b}\right)^\gamma + E\left(\tfrac{n}{b}\right)^\ell\right) + u'n^\ell \\ &= Dn^\gamma \underbrace{\left(a \cdot \tfrac{1}{b^\gamma}\right)}_{=1} + E\,a\,n^\ell \tfrac{1}{b^\ell} + u'n^\ell \end{aligned}$$

since $\gamma = \log_b a \Leftrightarrow b^\gamma = a$. The D-terms cancel, and the equation holds iff $\boldsymbol{E\left(1 - a/b^\ell\right) = u'}$.

Case	ℓ vs γ	b^ℓ vs a	E
$(+)$	$\ell > \gamma$	$b^\ell > a$	$E > 0$
$(-)$	$\ell < \gamma$	$b^\ell < a$	$E < 0$
$(=)$	$\ell = \gamma$	$b^\ell = a$	would need $u' = 0 \to$ no solution

- $(+)$ $T'(n) = Dn^\gamma + En^\ell$ with $E > 0$ solves (6). D is free: choose it big enough to fulfil (5). Then $T(n) \le \hat{T}(n) = T'(n - v) = O(n^\gamma) + O(n^\ell) = O(n^\ell)$.
- $(-)$ $E < 0$ solves (6), D is arbitrary. Choose D large enough to make T' monotone and to fulfil (5). Then $T(n) = O(n^\gamma) + O(n^\ell) = O(n^\gamma)$.
- $(=)$ Modified idea: $T'(n) = Dn^\gamma + En^\gamma \log_b n$. Substituting gives $Dn^\gamma + En^\gamma \log_b n - En^\gamma + u'n^\gamma$ on the right, so $E = u'$ works and $T(n) = O(n^\gamma \log n)$. (This calculation is not in the notes.)

The lower bounds follow in the same way with T^{low}.

2.8 Shifting the Domain

$$A(n) = 3\,A\left(\tfrac{n}{2} + 1\right) + 7n^2$$

The $+1$ inside the argument does not fit the master theorem. Set $B(n) := A(n - v)$ and choose v so that B has a clean recursion:

$$B(n) = A(n - v) = 3\,A\left(\tfrac{n-v}{2} + 1\right) + 7(n - v)^2 \stackrel{!}{=} 3\,B\left(\tfrac{n}{2}\right) + \dots$$

This needs $(n - v)/2 + 1 = n/2 - v$, i.e. $\boldsymbol{v = -2}$. So $B(n) = A(n + 2)$, and $A(n) = B(n - 2)$.

Clean recursion for $B(n)$:

$$B(n) = 3\,B\left(\tfrac{n}{2}\right) + 7(n + 2)^2$$

$\gamma = \log_2 3 \approx 1.585 < 2$ and $f = \Theta(n^2)$: Case III, so $B(n) = \Theta(n^2)$ and $A(n) = \Theta(n^2)$. It is the same trick as v in Step 5.

2.9 Shifting the Value

$$A(n) = 3\,A\left(\tfrac{n}{2}\right) + 7n^2 + 3$$

Set $C(n) := A(n) + W$:

$$C(n) = 3\left(C\left(\tfrac{n}{2}\right) - W\right) + 7n^2 + 3 + W = 3\,C\left(\tfrac{n}{2}\right) + 7n^2 + 3 - 2W$$

With $W = 3/2$ the constant disappears: $C(n) = 3C(n/2) + 7n^2$. In general $W = c/(a - 1)$ for an additive constant c. (The choice of W is not in the notes.)

Lecture of 26.10.2026 • Selection, Quickselect, median of medians

Divide and conquer also solves the **selection** problem: find the k-smallest element of a list, for example the median, in $O(n)$ instead of the $O(n \log n)$ of sorting. Quickselect with a random pivot needs expected linear time; this is a first application of the master theorem (Section 2.4) with $a = 1$. The median-of-medians pivot of Blum, Floyd, Pratt, Rivest and Tarjan (1973) makes it deterministic; its recurrence has two calls of different size, so we solve it by induction.

2.10 The Selection Problem

Definition (rank, selection). An element $a \in A$ has **rank** k if it is at position k when A is sorted in increasing order. **Selecting** means: given $k \in \{1, \dots, n\}$, find the k-smallest element of A. The **median** is the element of rank $\lfloor (n+1)/2 \rfloor$.

Example. $A = 3, 4, 7, 11, 12$: the median has rank $\lfloor 6/2 \rfloor = 3$, so it is 7. For $k = 2$ we get 4.

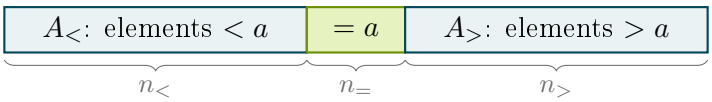
Computing the rank of a *given* element is easy: count the smaller elements, in $O(n)$. With equal elements the rank is not unique; an element can then have several possible ranks.

Goal: selection in $O(n)$. Sorting first costs $O(n \log n)$.

2.11 Quickselect

Quickselect is the selection counterpart of Quicksort: after the partition, Quicksort recurses into *both* parts, Quickselect only into the *one* that contains the answer.

Partitioning. Pick a pivot a and split A into $A_<$ (elements $< a$), $A_=$ (elements $= a$) and $A_>$ (elements $> a$):



with lengths $n_< + n_= + n_> = n$. This costs $O(n)$.

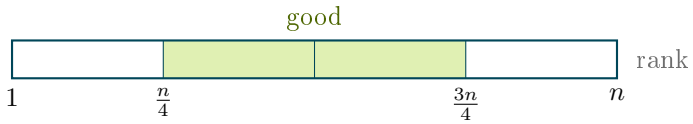
Recursion select(k, A):

1. if $k \le n_<$: return select$(k, A_<)$
2. else if $k \le n_< + n_=$: return a
3. else: return select$(k - n_< - n_=, A_>)$

Worst case. If a is always the smallest or largest element, the list shrinks by only 1 per step. The recursion depth is $n - 1$ and each level costs $\Theta(n)$, so the runtime is $\Theta(n^2)$.

2.12 Randomized Pivot

Definition (good partition). The pivot a gives a **good partition** if $n/4 \le \text{rank}(a) \le 3n/4$. If the rank is not unique, a is good if one of its possible ranks lies in this range.



- A uniformly random pivot is good with probability $\ge 1/2$, since half of all ranks lie in the middle range.
- After a good partition, $A_<$ and $A_>$ have at most $3n/4$ elements: the list shrinks to at most $\frac{3}{4}$ of its size.

Theoretical algorithm for the analysis. Each round has two phases:

1. Phase A: repeatedly sample a pivot until finding a good one.
2. Phase B: make the partition and proceed to $A_<$ or $A_>$, or terminate.

Phase A is a Bernoulli experiment with success probability $p \ge 1/2$, so it needs $1/p \le 2$ trials in expectation. Each trial costs $O(n)$, hence $E(\text{runtime of phase A}) = O(n)$.

Expected runtime. Let $T(n)$ be the maximal expected time over lists of size at most n, so T is monotone. Then

$$T(n) \le T\left(\left\lceil \tfrac{3n}{4} \right\rceil\right) + O(n).$$

Master theorem with $a = 1$, $b = 4/3$, $f(n) = \Theta(n)$: $\gamma = \log_{4/3} 1 = 0$ and the exponent of f is $1 > \gamma$, so Case III gives $\boldsymbol{T(n) = \Theta(n)}$. Unrolling gives the same: $cn \cdot (1 + \frac{3}{4} + \frac{9}{16} + \dots) = 4cn$.

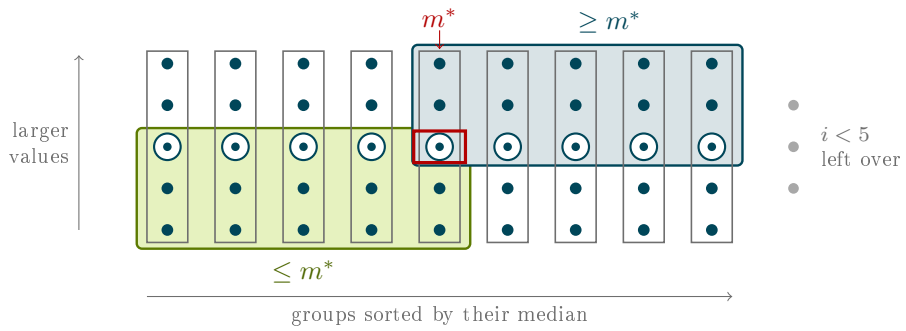
Remark (in practice). In practice we use *every* pivot, not only good ones: we simply partition and recurse. The analysis goes through as well and still gives expected time $O(n)$.

2.13 Deterministic Algorithm: Median of Medians

Blum, Floyd, Pratt, Rivest and Tarjan (1973) compute a good pivot deterministically:

1. Partition A into groups of 5 and determine the median $m_1, m_2, \dots, m_{\lfloor n/5 \rfloor}$ of each group.
2. Determine recursively the median m^* of the medians.
3. Use m^* as the pivot.

Step 1 costs $O(n)$, since every group has constant size. Step 2 is a recursive call on $\lfloor n/5 \rfloor \approx 0.2\,n$ medians.



Each column is a group of 5, larger values on top, its median circled; the columns are ordered by their medians, so m^* (red) is the median of the middle column. Everything in the green box is $\leq m^*$, everything in the blue box $\geq m^*$. The sorting is only for the picture; the algorithm sorts nothing.

How much do we get rid of? At least half of the $\lfloor n/5 \rfloor$ medians are $\leq m^*$, and each brings 3 elements $\leq m^*$ (itself and the two below it). The same holds for $\geq m^*$. So we get rid of $\geq 3 \cdot \lfloor n/5 \rfloor \cdot \frac{1}{2} \approx 0.3\,n$ elements, and the size after step 3 is at most about $0.7\,n$.

Exact bound. Assume we go to $A_>$. Write $n = 5\ell + i$ with $i \in \{0, \ldots, 4\}$. At least $3\ell/2$ elements are $\leq m^*$, and none of them is in $A_>$:

$$n_> \leq n - 3 \cdot \frac{\ell}{2} = n - 3 \cdot \frac{n-i}{10} = \frac{7n}{10} + \underbrace{\frac{3i}{10}}_{<2}.$$

Hence $n_> \leq 7n/10 + 2$, and by symmetry the same holds for $n_<$.

Linear runtime. Let $T(n)$ be the worst-case runtime on lists of size at most n (so T is monotone). With $\lfloor 7n/10 \rfloor + 3$ as a safe integer bound for the second call:

$$T(n) \leq T\left(\left\lfloor \tfrac{n}{5} \right\rfloor\right) + T\left(\left\lfloor \tfrac{7n}{10} \right\rfloor + 3\right) + cn.$$

The master theorem does not apply (two calls of different size), so we use induction.

Theorem 2.3 (Linear-time selection). There is a constant D with $T(n) \leq Dn$ for all $n \geq 1$.

Proof. Choose $D \geq 20c$, and D big enough that $T(n) \leq Dn$ for $1 \leq n \leq 60$. For $n > 60$ both arguments are smaller than n, so by induction:

$$\begin{aligned} T(n) &\leq D\left\lfloor \tfrac{n}{5} \right\rfloor + D\left(\left\lfloor \tfrac{7}{10}n \right\rfloor + 3\right) + cn \\ &\leq D\,\tfrac{n}{5} + D\,\tfrac{7}{10}n + \tfrac{n}{20}D + cn \qquad \left(3 \leq \tfrac{n}{20} \text{ for } n \geq 60\right) \\ &= Dn\,(0.2 + 0.7 + 0.05) + cn \\ &\leq Dn\,(0.95 + 0.05) = Dn \qquad (cn \leq 0.05\,Dn \text{ since } D \geq 20c). \end{aligned}$$

■

The key is $\frac{1}{5} + \frac{7}{10} < 1$: both subproblems together are smaller than the original, so the work per level shrinks geometrically.

Notes on Corrections

These deviate from the handwritten notes or the official slides and should be checked against the lecture.

From the notes of 19.10.2026:

- Induction base Mergesort: $A \geq C_0/2$ (not $C_0/4$).
- Karatsuba: $b := x_H \cdot y_H$ (notes say $x_H \cdot x_H$).
- Strengthened claim: $T(n) \leq A \cdot n^\gamma - Dn$ with minus (notes and slides say $+Dn$).
- $\log_2 3 \approx 1.585$ (slides: ≈ 1.7).
- Master theorem: $b > 1$, $\gamma \geq 0$, Case 2 with Θ, Case 3 with Ω (slides: Θ, $\gamma > 0$).

From the notes of 22.10.2026:

- $\gamma = \log_b a \geq 0$ (notes: > 0); $a = 1$ gives $\gamma = 0$.
- Case III: the notes write $f = \Theta(n^{\gamma+\delta})$ or the regularity condition. Both are correct; the regularity condition alone even implies $f = \Omega(n^{\gamma+\delta})$. Only with the weaker $f = \Omega(n^{\gamma+\delta})$ is the regularity condition needed in addition (see 2.4).
- Regularity example: with $\lceil n/2 \rceil \leq (n+1)/2$ the bound $1/2$ holds from $n \geq 5$; the notes' estimate $2^{-n/2+2}$ reaches $1/2$ only from $n \geq 6$.
- Gap example: the solution is $\Theta(n \log^2 n)$ (notes: $n \log_2 n$), as the extended Case II confirms.
- Shifting the domain: $v = -2$, so $B(n) = A(n+2)$ and $A(n) = B(n-2)$ (notes: $v = 2$, $A(n) = B(n+2)$). With $v = 2$ the argument becomes $n/2 + 2$, not $n/2$.
- Step 2: "enforcing a bigger n_0" (notes: "a bigger a").

From the notes of 26.10.2026:

- Goal: "Selecting in $O(n)$" (notes: "Sorting in $O(n)$"); sorting by comparisons needs $\Omega(n \log n)$.
- Quickselect, second case: return a if $k \leq n_< + n_=$ (notes: $k \leq n_<$). Third case: recurse into $A_>$ otherwise, i.e. $k > n_< + n_=$ (notes: $k \leq n_< + n_=$).
- Good partition: the size decreases *to* at most $\frac{3}{4}$ of $|A|$, i.e. *by* at least $\frac{1}{4}$.
- Phase A terminates after ≤ 2 iterations in expectation (notes: "often").
- Master theorem for Quickselect: $b = 4/3$ and $\gamma = 0$; the exponent of f is $1 > \gamma$, hence Case III.
- Exact bound: $n - 3 \cdot \ell/2 = n - 3 \cdot (n-i)/10$ (notes: $(n-1)/10$), and the conclusion is about $n_>$ (we assumed $A_>$); $n_<$ follows by symmetry.
- $\lfloor 7n/10 \rfloor + 2$ already suffices as recursion size; the $+3$ in the notes is a safe overestimate.
- Added: T is the maximum over all lists of size at most n, so T is monotone; both recurrences need this.
- Final step: $Dn\,(0.95 + 0.05) = Dn$ (notes: $0.95 + 0.5$). The conditions are $n \geq 60$ for $3D \leq \frac{n}{20}D$ and $D \geq 20c$ for $cn \leq \frac{n}{20}D$.